

Visualisierung kinematischer Kennwerte

Philipp Mikos

April 2025

1 Einleitung

Die Grundlagen der Kinematik stellen einen wesentlichen Bestandteil der Robotik dar und sind sowohl für mobile als auch für stationäre Roboter von zentraler Bedeutung. Während es bei stationären Robotern häufig darum geht, die Position des Endeffektors präzise zu berechnen, konzentriert sich die Kinematik bei mobilen Robotern vor allem darauf, deren Position zuverlässig bestimmen zu können. Ein fundiertes Verständnis der erforderlichen kinematischen Modelle ist unerlässlich, um diese Größen korrekt zu berechnen. Siegwart, Nourbakhsh und Scaramuzza 2011, S. 57

Im Rahmen dieser Projektarbeit wird ein Framework entwickelt, das die Visualisierung kinematischer Kennwerte ermöglicht. Das Ziel dieses Frameworks ist es, unterstützend in der Lehre eingesetzt zu werden, um das Verständnis der Kinematik zu fördern und die praktische Anwendung dieser Konzepte zu erleichtern. Ein besonderer Fokus dieser Arbeit liegt auf der Rotation von Körpern im Raum. Dabei wird insbesondere erläutert, was unter Eulerwinkeln sowie Roll-Pitch-Yaw-Winkeln zu verstehen ist und worin die Unterschiede zwischen beiden Darstellungen bestehen. Darüber hinaus werden auch Rotationsmatrizen sowie Quaternionen behandelt, um die verschiedenen Ansätze zur Beschreibung von Rotationen umfassend darzustellen.

2 Theoretischer Hintergrund

2.1 Darstellung der Position und Orientierung eines Roboters

Ein Roboter wird in der Regel als ein unverformbarer, fester Körper (Rigid Body) betrachtet, der sowohl eine bestimmte Position als auch eine Orientierung im Raum besitzt. Die Position beschreibt den

Abstand und die Verschiebung zwischen dem globalen Weltkoordinatensystem und dem körpereigenen Koordinatensystem des Roboters. Die Orientierung hingegen gibt den Unterschied in der Winkelausrichtung zwischen dem Weltkoordinatensystem und dem lokalen Koordinatensystem des Roboters an. Die Kombination aus Position und Orientierung eines Körpers wird als Pose bezeichnet. Die Transformation von einer Pose in eine andere kann intuitiv als Bewegung interpretiert werden. Diese Transformation setzt sich aus einer Translation (Verschiebung) und einer Rotation (Drehung) zusammen. (vgl. Robotics, Vision and Control, S. 24–27) Speziell bei radgetriebenen Robotern wird die Pose auf einer Ebene beschrieben. In diesem Fall besteht die Pose aus den X- und Y-Koordinaten im globalen Koordinatensystem sowie dem Winkel zwischen dem globalen und dem körpereigenen Koordinatensystem. Damit lässt sich die Pose als ein Vektor aus diesen drei Elementen darstellen. Siegart, Nourbakhsh und Scaramuzza 2011, S. 59

2.2 Euler-Winkel und Roll-Pitch-Yaw-Winkel

Euler's Rotationstheorem besagt, dass zwei beliebige unabhängige, orthonormale Koordinatensysteme durch eine Abfolge von maximal drei Rotationen um Koordinatenachsen miteinander in Beziehung gesetzt werden können, wobei keine zwei aufeinanderfolgenden Rotationen um die gleiche Achse erfolgen dürfen (vgl. Kuipers, 1999). Das bedeutet, dass jede Rotation zwischen zwei Koordinatensystemen im dreidimensionalen Raum durch eine Sequenz von drei Rotationswinkeln beschrieben werden kann, wobei jeder Winkel einer bestimmten Achse zugeordnet ist. Die Rotationen werden nacheinander ausgeführt: Zunächst erfolgt eine Rotation des Weltkoordinatensystems um eine Achse, wodurch ein neues Koordinatensystem entsteht. Anschließend wird dieses neue System um eine seiner eigenen Achsen rotiert, gefolgt von einer dritten Rotation um eine Achse des wiederum neu entstandenen Systems.

Insgesamt gibt es zwölf eindeutige Rotationsabfolgen. Sechs davon beinhalten Wiederholungen derselben Achse (aber nicht unmittelbar hintereinander), beispielsweise XYX , XZX , YXY , YZY , ZZX oder ZZY . Die anderen sechs Rotationsabfolgen verwenden jeweils alle drei Achsen einmal, beispielsweise XYZ , XZY , YXZ , ZXY oder ZYX .

Der Begriff Euler-Winkel wird häufig verwendet, um jede Art von dreifacher Rotationsdarstellung zu bezeichnen. Diese Bezeichnung ist jedoch nicht präzise, da – abhängig von der Reihenfolge der Rotationen – verschiedene Darstellungen entstehen. Deshalb muss die genaue Rotationsabfolge angegeben werden, auch wenn innerhalb bestimmter technischer Fachgebiete meist eine stillschweigende Konvention herrscht.

Eine besondere Rolle spielen die sogenannten Roll-Pitch-Yaw-Winkel, bei denen es zwei gebräuchliche Rotationssequenzen gibt: ZYX und XYZ. In der mobilen Robotik wird typischerweise die ZYX-Sequenz verwendet, während bei stationären Roboter manipulatoren oft die XYZ-Sequenz Anwendung findet. Corke 2023, S. 48–51

2.3 Rotationsmatrizen

Eine Rotationsmatrix ist eine spezielle Art von Matrix, die ein Koordinatensystem um einen Winkel θ dreht und es dadurch in ein neues Koordinatensystem transformiert. Die Spalten einer Rotationsmatrix können dabei als die transformierten Einheitsvektoren des ursprünglichen Koordinatensystems interpretiert werden.

Rotationsmatrizen sind stets orthonormal, das bedeutet, ihre Spalten (bzw. Zeilen) stehen senkrecht aufeinander und haben die Länge Eins. Daraus folgt die wichtige Eigenschaft:

$$R^{-1} = R^T$$

wobei R^{-1} die Inverse und R^T die Transponierte der Matrix R ist. Ein weiteres charakteristisches Merkmal von Rotationsmatrizen ist, dass ihre Determinante stets den Wert 1 hat.

Rotationsmatrizen sind außerdem Elemente der sogenannten speziellen orthogonalen Gruppe der Dimension 2, die als $SO(2)$ bezeichnet wird. dies schreiben wir auch als

$$R \in SO(2) \subset \mathbb{R}^{2 \times 2}$$

Die spezielle orthogonale Gruppe ist eine Gruppe bezüglich der Matrixmultiplikation. Das bedeutet, dass das Produkt zweier Rotationsmatrizen wieder eine Rotationsmatrix ist und somit ebenfalls zur Gruppe gehört. Ebenso ist die Inverse jeder Rotationsmatrix ebenfalls Teil der Gruppe. Corke 2023, S. 33

2.4 Quaternionen und Unit-Quaternionen

Quaternionen wurden 1843 von Sir William Rowan Hamilton entdeckt. Sie erweitern die komplexen Zahlen und bestehen aus einem Skalar- und einem Vektoranteil:

$$q = s + v_x i + v_y j + v_z k$$

wobei i , j und k besondere Einheiten mit $i^2 = j^2 = k^2 = ijk = -1$ sind. Quaternionen bilden einen Vektorraum: Man kann sie addieren, skalieren, multiplizieren (Hamilton-Produkt) und konjugieren.

Die Multiplikation ist nicht kommutativ. Der Betrag eines Quaternions ist:

$$||q|| = \sqrt{s^2 + v_x^2 + v_y^2 + v_z^2}$$

Unit-Quaternionen sind Quaternionen mit Betrag 1. Sie werden häufig verwendet, um Rotationen im Raum darzustellen. Eine Rotation um eine Achse $\hat{v}$ mit Winkel θ wird als

$$q = \cos(\theta/2) + \hat{v} \sin(\theta/2)$$

bezeichnet.

Quaternionen bieten bei der Darstellung von Rotationen mehrere Vorteile. Sie ermöglichen eine recheneffiziente Kombination von Rotationen und vermeiden dabei Probleme wie den sogenannten Gimbal Lock, der bei der Verwendung von Euler-Winkeln auftreten kann. Zudem sind Quaternionen kompakter als Rotationsmatrizen und benötigen weniger Speicherplatz und Rechenleistung, was sie besonders attraktiv für Anwendungen in Bereichen wie Robotik, Computer Vision, Computergrafik und Navigation macht. Corke 2023, S. 59–61

2.5 Differentialantrieb

Ein mobiler Roboter mit Differentialantrieb besitzt zwei angetriebene Räder, jeweils eines auf jeder Seite des Roboters. Größere Roboter können zusätzlich über ein oder mehrere weitere Räder auf jeder Seite verfügen, die entweder passiv sind oder mechanisch mit den aktiven Rädern gekoppelt werden, sodass sie sich einen Motor teilen. Die Räder sind starr am Roboter befestigt.

Die Steuerung des Roboters erfolgt durch die unterschiedliche Antriebsgeschwindigkeit der Räder. Drehen sich beide angetriebenen Räder mit gleicher Geschwindigkeit, bewegt sich der Roboter geradeaus. Rotieren die Räder hingegen mit unterschiedlichen Geschwindigkeiten, führt dies zu einer Drehbewegung des Roboters.

Bei Robotern mit Differentialantrieb wird üblicherweise davon ausgegangen, dass es sich bei den verbauten Rädern um Standardräder handelt. Das bedeutet, dass sich die Räder – und somit der gesamte Roboter – ausschließlich in x-Richtung bewegen können, während Bewegungen oder ein Rutschen in y-Richtung nicht möglich sind. Zudem wird angenommen, dass jedes Rad nur einen einzigen Kontaktpunkt mit der Ebene besitzt. Corke 2023, S. 140–141 Hertzberg, Lingemann und Nüchter 2012, S. 107

2.6 JupyterLab

JupyterLab ist eine leistungsstarke und erweiterbare Umgebung zur Erstellung und Bearbeitung von sogenannten computational notebooks. Es ist Teil des größeren Project Jupyter, das sich zum Ziel gesetzt hat, Werkzeuge und Standards für das interaktive Arbeiten mit Rechen-Notizbüchern zu entwickeln.

Ein computational notebook ist ein Dokument, das Computer-Code, Freitext-Beschreibungen, Daten, grafische Darstellungen (z.B. Diagramme, 3D-Modelle) sowie interaktive Bedienelemente miteinander kombiniert. In Verbindung mit einem Editor wie JupyterLab ermöglichen diese Notebooks eine schnelle, interaktive Entwicklung, die sich besonders gut für das Prototyping von Code, die Datenexploration, die Visualisierung und das Teilen von Ideen eignet.

JupyterLab ist eng verwandt mit anderen Anwendungen unter dem Dach von Project Jupyter, wie zum Beispiel Jupyter Notebook und Jupyter Desktop. Im Vergleich zu Jupyter Notebook bietet JupyterLab jedoch eine fortschrittlichere, funktionsreichere und individuell anpassbare Benutzererfahrung. Jupyter
2024

3 Anforderungen

Das im Rahmen dieser Projektarbeit entwickelte Framework ist für die Nutzung in der JupyterLab-Umgebung konzipiert und daher in der Programmiersprache Python implementiert. Ziel des Frameworks ist es, Transformationen von Körpern im dreidimensionalen Raum zu visualisieren. Die Visualisierungen sollen bevorzugt direkt innerhalb von JupyterLab ausgeführt werden. Darüber hinaus ist das Framework so gestaltet, dass es die komfortable Erstellung von Übungsaufgaben unterstützt und erleichtert.

4 Technische Umsetzung

Um eine einfache Installation und Nutzung des Frameworks zu gewährleisten, ist es erforderlich, Versionskonflikte zwischen den verwendeten Abhängigkeiten zu vermeiden. Zu diesem Zweck wird das Werkzeug Poetry eingesetzt. Poetry ermöglicht die Verwaltung aller benötigten Python-Bibliotheken in festen Versionen und erstellt eine isolierte virtuelle Umgebung, in der alle Abhängigkeiten konsistent installiert sind. Dadurch wird sichergestellt, dass das Framework unabhängig von der Zielumgebung

stabil und reproduzierbar ausgeführt werden kann.

Eine der zentralen Bibliotheken innerhalb des Projekts ist NumPy, eine weit verbreitete Python-Bibliothek für numerische Berechnungen. NumPy wird hauptsächlich für erweiterte mathematische Operationen im Bereich der linearen Algebra eingesetzt.

Darüber hinaus ist pythreejs als weitere wichtige Abhängigkeit integriert. Es handelt sich hierbei um eine Python-Wrapper-Bibliothek für die JavaScript-Bibliothek three.js, die auf WebGL basiert. pythreejs ermöglicht die Darstellung und Interaktion mit 3D-Geometrien innerhalb eines WebGL-Renderers. Dieser Renderer kann nahtlos in der JupyterLab-Umgebung eingebettet und somit direkt in einem Jupyter Notebook visualisiert und gesteuert werden.

Durch den Einsatz dieser Technologien wird eine effiziente und plattformunabhängige Darstellung sowie Manipulation komplexer geometrischer Strukturen innerhalb des Frameworks ermöglicht.

Die Bibliothek des Frameworks besteht aus einer zentralen Datei, util.py. In dieser Datei sind sämtliche Kernfunktionen implementiert, die zur Erstellung, Transformation und Animation geometrischer Formen benötigt werden. Darüber hinaus enthält util.py eine Reihe von Hilfsfunktionen, die unter anderem die Umrechnung zwischen verschiedenen Darstellungsformen von Rotationen ermöglichen sowie die Erstellung von Koordinatensystemen unterstützen.

Literatur

Corke, Peter (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python*. eng. Third edition. Bd. 146. Springer Tracts in Advanced Robotics. Cham: Springer International Publishing AG. ISBN: 3031064682.

Hertzberg, Joachim, Kai Lingemann und Andreas Nüchter (2012). *Mobile Roboter : eine Einführung aus Sicht der Informatik*. ger. eXamen.press. Berlin [u.a: Springer. ISBN: 9783642017254.

Jupyter, Project (2024). *JupyterLab Documentation*. URL: <https://jupyterlab.readthedocs.io/en/stable/> (besucht am 14.04.2025).

Siegwart, Roland, Illah Reza Nourbakhsh und Davide Scaramuzza (2011). *Introduction to autonomous mobile robots*. eng. 2. ed. Intelligent robotics and autonomous agents. Cambridge, Mass. [u.a: MIT Press. ISBN: 9780262015356.